

INFO04
Curso: UFCD 10810
UFCD/Módulo/Temática: UFCD 10810 - Fundamentos do desenvolvimento de modelos analíticos em Python
Ação: 10810_1L
Formador/a: Sandra Liliana Meira de Oliveira
Data:
Nome do Formando/a:

Modelos Não -Supervisionados:

1. Introdução à Aprendizagem Não-Supervisionada

1. Introdução à Aprendizagem Não-Supervisionada

O que é Aprendizagem Não-Supervisionada?

Na aprendizagem não-supervisionada, os algoritmos procuram **descobrir padrões e relações complexas nos dados sem supervisão explícita**, ou seja, sem rótulos ou categorias predefinidas. Em vez de aprender a partir de um conjunto de dados anotado, como ocorre na aprendizagem supervisionada, o modelo procura **identificar estruturas latentes** ou agrupamentos naturais dentro dos dados. Esta abordagem é especialmente útil quando os dados são abundantes mas não rotulados — o que é comum em muitos contextos do mundo real.

A aprendizagem não-supervisionada é frequentemente tratada como um **problema de inferência estatística não guiada**, onde se parte da hipótese de que os dados foram gerados por um processo com estrutura subjacente desconhecida. O objetivo é inferir essa estrutura com base em métricas de similaridade, distância ou densidade.

Aplicações Típicas:

- **Segmentação de clientes** (ex.: marketing personalizado com base em comportamento de compra)
- **Agrupamento de documentos semelhantes** (ex.: organização automática de artigos científicos)
- **Compressão de dados** (ex.: redução da dimensionalidade mantendo características relevantes)

- **Deteção de anomalias** (ex.: identificação de padrões fora do comum em sistemas bancários ou industriais)

2. Conceito de Clustering (Agrupamento)

O que é Clustering?

Clustering é uma das tarefas especialmente fundamentais em aprendizagem não-supervisionada. Consiste em **agrupar instâncias semelhantes entre si**, de forma que os objetos no mesmo grupo (ou cluster) tenham **maior similaridade entre si do que com objetos de outros grupos**.

Este processo baseia-se na premissa de que existe uma estrutura implícita nos dados que pode ser desvendada por meio de algoritmos capazes de explorar critérios de proximidade, densidade, conectividade ou distribuição probabilística.

Medidas de Similaridade e Distância

O conceito de similaridade é central no clustering. Entre as medidas mais comuns estão:

- **Distância Euclidiana:** adequada para variáveis quantitativas contínuas.
- **Distância de Manhattan:** soma das diferenças absolutas.
- **Cosine Similarity:** usada principalmente para dados textuais ou vetores de alta dimensionalidade.
- **Distância de Mahalanobis:** leva em consideração a correlação entre variáveis.

Exemplo intuitivo (sem fórmulas): se desenharmos pontos num plano, os que estão próximos devem cair no mesmo cluster — tal como pessoas próximas fisicamente em festas tendem a formar grupos.

3. Métodos de Clustering

Os métodos de clustering dividem-se em categorias principais, sendo as mais importantes:

- **Clustering de Partição** (ex.: K-Means)
- **Clustering Hierárquico** (ex.: Aglomerativo)
- **Clustering baseado em densidade** (ex.: DBSCAN)
- **Modelos probabilísticos** (ex.: Mistura de Gaussianas – GMM)

3.1 Clustering Hierárquico

- Constrói uma **hierarquia de clusters**, geralmente representada por um **dendrograma** (Um Dendrograma (dendro = árvore) é um tipo específico de diagrama ou representação icónica que organiza determinados fatores e variáveis - <https://pt.wikipedia.org/wiki/Dendrograma>).
- No método **aglomerativo** (bottom-up): cada ponto começa como um cluster individual, e, a cada iteração, os dois clusters mais próximos são fundidos.
- Este processo continua até restar um único cluster.

Vantagens:

- Não exige escolha prévia do número de clusters.
- Representação hierárquica útil para análises exploratórias.

Limitações:

- Custo computacional elevado para grandes volumes de dados.
- Sensível à definição da métrica de distância.

3.2 K-Means

- Algoritmo de partição que assume que os clusters têm forma aproximadamente esférica e semelhante densidade.
- Funcionamento:
 1. Definir o número de clusters (K)
 2. Inicializar K centróides (que representam os centros dos clusters) aleatórios
 3. Atribuir cada ponto ao centróide mais próximo
 4. Recalcular centróides como média dos pontos em cada cluster
 5. Repetir até convergência

Considerações:

- A escolha de **K** é crítica; o **método do cotovelo** pode auxiliar na definição ótima. Dizer quantos clusters queremos encontrar é um desafio principalmente quando estamos a lidar com aprendizagem não supervisionado. Para resolver essa questão existe um método conhecido como Método Cotovelo. A ideia é básica, definir a melhor quantidade de clusters que podem ser encontrados, mesmo sem saber a resposta antecipadamente (<https://medium.com/pizzadedados/kmeans-e-metodo-do-cotovelo-94ded9fdf3a9>).

- K-Means é sensível a outliers e à escala dos dados (normalização é recomendada).

3.3 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- Algoritmo baseado em densidade que identifica clusters como regiões de alta densidade separadas por regiões de baixa densidade.
- **Não requer a definição de K** (número de clusters).
- Dois parâmetros principais:
 - eps: raio de vizinhança
 - min_samples: número mínimo de pontos para formar uma região densa

Vantagens:

- Identifica clusters de forma arbitrária (não assume forma esférica)
- Lida bem com **outliers** e **ruído**

Limitações:

- Sensível à escolha dos parâmetros eps e min_samples
- Menos eficaz em dados de densidade variável

3.4 GMM (Gaussian Mixture Models)

- Abordagem probabilística: assume que os dados são gerados a partir de uma mistura de distribuições gaussianas.
- Cada cluster é modelado como uma **distribuição normal multivariada**.
- Utiliza o algoritmo **EM (Expectation-Maximization)** para ajustar os parâmetros.

Vantagens:

- Mais flexível do que K-Means (permite clusters com diferentes tamanhos e formas)
- Permite inferência probabilística (ex: probabilidade de pertença a cada cluster)

Limitações:

- Pressupõe que os dados seguem distribuições normais
- Sensível à inicialização e ao número de componentes

4. Clustering como Modelo Não-Supervisionado

Embora o clustering seja frequentemente considerado uma tarefa exploratória, pode ser formalizado como um **modelo não-supervisionado** no contexto mais amplo da aprendizagem automática:

- **Entrada:** matriz de características
- **Saída:** vetor de etiquetas de clusters

Modelos baseados em clustering podem ser integrados em sistemas maiores:

- **Pré-processamento** para modelos supervisionados (ex.: criar novas features baseadas em grupos)
- **Compressão de dados** com redução de complexidade
- **Extração de protótipos** (representantes de cada grupo)
- **Deteção de estruturas latentes** que não são evidentes nos dados brutos

Na investigação científica e em projetos, o clustering é amplamente utilizado como:

- Técnica exploratória antes de modelagem supervisionada
- Ferramenta para descoberta de novos padrões
- Forma de validação cruzada

Clustering vai além de métodos clássicos como K-Means e Clustering Hierárquico. Técnicas mais avançadas como **DBSCAN** e **GMM** permitem:

- Lidar com **clusters de forma arbitrária**, não esféricos;
- Lidar com **outliers e ruído** (DBSCAN);
- Fazer **inferência probabilística** e modelar **estruturas com diferentes covariâncias** (GMM);
- Adaptar-se melhor a **distribuições heterogéneas e espaços de alta dimensionalidade**.

Estas técnicas avançadas são essenciais para aplicações reais em que a simplicidade de K-Means não é suficiente — por exemplo, deteção de anomalias, segmentação de utilizadores com padrões complexos, ou agrupamento de textos ou imagens.

5. Exemplos Práticos com o Dataset Iris e Dados Sintéticos

Exemplo 1: Clustering Hierárquico com Scikit-learn

Nota: O método hierárquico aglomerativo segue uma abordagem **bottom-up** (de baixo para cima), onde:

- Cada ponto começa como um cluster individual;
- A cada iteração, os dois clusters mais próximos são fundidos;
- O processo continua até atingir o número de clusters desejado ou formar um único grande cluster.

```
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
import pandas as pd

# Carregar dados
iris = load_iris()
data = iris.data
labels_true = iris.target

# Criar modelo de clustering hierárquico
# n_clusters: número de grupos a formar
# linkage='ward': método de ligação baseado em variância mínima

model = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels_pred = model.fit_predict(data)

# Comparar com classes reais
pd.crosstab(labels_pred, labels_true)
```

Explicação:

- AgglomerativeClustering executa um método bottom-up de agrupamento.
- n_clusters=3: define o número de grupos finais desejados.
- linkage='ward': minimiza a variância total dentro de cada cluster.
- A tabela cruzada ajuda a avaliar visualmente a correspondência entre os clusters e as classes verdadeiras do dataset.

Exemplo 2: K-Means no Iris

Nota: Este exemplo reutiliza os dados carregados no Exemplo 1. Se estiveres a executar este bloco de forma isolada, certifica-te de que os dados estão carregados com:

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.datasets import load_iris
iris = load_iris()
data = iris.data

# Criar modelo K-Means
# n_clusters: número de clusters desejado
# random_state: controla a aleatoriedade da inicialização

kmeans = KMeans(n_clusters=3, random_state=42)
labels_kmeans = kmeans.fit_predict(data)

# Mostrar centróides
print("Centróides:", kmeans.cluster_centers_)

# Avaliar com coeficiente de silhueta
sil_score = silhouette_score(data, labels_kmeans)
print(f"Silhouette Score: {sil_score:.2f}")
```

Explicação:

- `n_clusters=3`: especifica o número de grupos.
- `fit_predict`: ajusta o modelo e obtém as etiquetas dos clusters.
- `cluster_centers_`: devolve as coordenadas dos centróides.
- `silhouette_score`: mede quão bem separados estão os clusters (varia entre -1 e 1).

Exemplo 3: Visualização com PCA

Nota: Este exemplo pressupõe que os dados (`data`) e os rótulos dos clusters (`labels_kmeans`) foram gerados previamente, como no Exemplo 2.

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
iris = load_iris()
data = iris.data
kmeans = KMeans(n_clusters=3, random_state=42)
labels_kmeans = kmeans.fit_predict(data)
```

```

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Redução de dimensionalidade para visualização
pca = PCA(n_components=2)
data_2d = pca.fit_transform(data)

# Visualizar clusters do K-Means
plt.scatter(data_2d[:, 0], data_2d[:, 1], c=labels_kmeans, cmap='viridis')
plt.title("Clusters encontrados pelo K-Means")
plt.xlabel("Componente 1")
plt.ylabel("Componente 2")
plt.show()

```

Explicação:

- PCA reduz as 4 dimensões do Iris para 2 componentes principais.
- Permite visualizar os agrupamentos de forma mais intuitiva.
- As cores representam os clusters encontrados.

Exemplo 4: DBSCAN com Dataset Iris (2 features para visualização)

```

from sklearn.datasets import load_iris
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Carregar e preparar os dados
iris = load_iris()
data = iris.data[:, :2] # Usar apenas 2 features para visualização
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# Aplicar DBSCAN
# eps: raio da vizinhança
# min_samples: número mínimo de pontos para formar um cluster
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels_db = dbscan.fit_predict(data_scaled)

# Visualizar os clusters
plt.scatter(data_scaled[:, 0], data_scaled[:, 1], c=labels_db, cmap='Set1')
plt.title("Clusters encontrados por DBSCAN")
plt.xlabel("Feature 1 (normalizada)")
plt.ylabel("Feature 2 (normalizada)")
plt.show()

# Mostrar etiquetas encontradas

```

```
print("Etiquetas encontradas:", set(labels_db))
```

Explicação:

`eps` controla o raio de vizinhança e `min_samples` o mínimo de pontos para formar uma região densa. O rótulo `-1` indica pontos classificados como ruído.

Exemplo 5: Gaussian Mixture Model (GMM) no Dataset Iris

```
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
from sklearn.datasets import load_iris

# Carregar dados
iris = load_iris()
X = iris.data
true_labels = iris.target

# Ajustar modelo GMM
gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
gmm.fit(X)
pred_labels = gmm.predict(X)

# Avaliação com índice Rand ajustado
ari = adjusted_rand_score(true_labels, pred_labels)
print(f"Adjusted Rand Index (ARI): {ari:.2f}")
```

Explicação:

O GMM modela clusters como distribuições normais. O `ARI` avalia a sobreposição entre os clusters encontrados e as classes reais.

Exemplo 6: Explorar DBSCAN com diferentes valores de `eps`

```
from sklearn.datasets import load_iris
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# Carregar e preparar os dados
iris = load_iris()
data = iris.data[:, :2] # Usar apenas duas features
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# Testar diferentes valores de eps
for eps_val in [0.3, 0.5, 0.7]:
    dbscan = DBSCAN(eps=eps_val, min_samples=5)
```

```
labels = dbscan.fit_predict(data_scaled)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print(f"eps = {eps_val}: clusters encontrados = {n_clusters}")
```

Explicação:

- eps define o raio da vizinhança local para cada ponto.
- min_samples=5: mínimo de pontos para formar um cluster denso.
- O número de clusters encontrados muda com eps, afetando a sensibilidade a ruído e a separação de grupos.

Exemplo 7: GMM aplicado a dados sintéticos

```
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt

# Gerar dados sintéticos gaussianos
X_synth, y_synth = make_blobs(
    n_samples=300,      # número total de amostras
    centers=3,          # número de clusters
    cluster_std=0.60,   # dispersão dos clusters
    random_state=0      # reprodutibilidade
)

# Criar e ajustar modelo GMM
gmm_synth = GaussianMixture(n_components=3, random_state=0)
gmm_synth.fit(X_synth)

# Obter previsões dos clusters
y_pred = gmm_synth.predict(X_synth)

# Visualizar resultados
plt.scatter(X_synth[:, 0], X_synth[:, 1], c=y_pred, cmap='viridis')
plt.title("Clusters encontrados pelo GMM (dados sintéticos)")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Explicação:

Este exemplo mostra como o GMM consegue identificar clusters com forma e densidade semelhantes às gaussianas, sendo ideal para dados com estas características.

NOTA: **Dados sintéticos** são conjuntos de dados **artificiais**, ou seja, **gerados manualmente ou por algoritmos**, em vez de serem recolhidos do mundo real. São amplamente utilizados para fins de:

- **Teste e desenvolvimento de algoritmos;**
- **Simulação de cenários controlados;**
- **Validação de modelos em ambiente seguro e previsível.**

Por que usar dados sintéticos?

1. **Permitem controlar totalmente a estrutura dos dados** (ex.: número de clusters, dispersão, dimensões).
2. **Facilitam a visualização e compreensão de algoritmos de aprendizagem.**
3. **Evita problemas legais ou de privacidade**, já que não envolvem dados reais de pessoas.

Exemplo 8: Clustering com Dataset do Kaggle – Wine Quality

Neste exemplo, utilizamos um dataset real retirado do Kaggle: o Wine Quality Dataset, que contém atributos físico-químicos de vinhos e a sua qualidade sensorial avaliada por especialistas.

Fonte do dataset: <https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Carregar dataset local (assumindo que está em CSV na mesma pasta)
# Tentar carregar o dataset com verificação

df = pd.read_csv("winequality-red.csv")

# Pré-processamento: normalização
# Utilizamos todas as colunas exceto 'quality' como variáveis preditoras
features = df.columns[:-1] # todas as colunas exceto 'quality'
X = df[features].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Clustering com K-Means (escolhendo 3 clusters por hipótese)
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X_scaled)

# Redução de dimensionalidade para visualização
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Visualizar clusters
plt.figure(figsize=(8, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', alpha=0.6)
plt.title("Clusters de Vinho - Dataset Wine Quality (K-Means)")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.show()
```

Explicação:

- O dataset possui 11 variáveis contínuas, como teor alcoólico, acidez, pH, etc.
- Aplicamos normalização para igualar a escala entre atributos.
- Usamos PCA para reduzir para 2 dimensões e facilitar a visualização.
- Este tipo de análise pode ser útil para **segmentar vinhos com características semelhantes**, por exemplo para posicionamento no mercado.

8. Como Utilizar Modelos Treinados com Novos Dados

Depois de treinar um modelo de clustering, como K-Means ou GMM, é possível aplicar o modelo a **novas observações** para descobrir a que grupo elas pertencem. Abaixo está o procedimento passo a passo:

Passo 1: Suponha um novo exemplo

```
# Exemplo: novo vinho com características medidas
novo_vinho = [[7.4, 0.70, 0.00, 1.9, 0.076, 11.0, 34.0, 0.9978, 3.51, 0.56, 9.4]]
```

Passo 2: Normalizar com o mesmo scaler

```
# Normalizar usando o mesmo objeto scaler ajustado nos dados de treino
novo_vinho_scaled = scaler.transform(novo_vinho)
```

Passo 3: Prever o cluster com K-Means

```
# Prever o cluster para o novo vinho
cluster = kmeans.predict(novo_vinho_scaled)
print(f"O novo vinho pertence ao cluster: {cluster[0]}")
```

GMM (Gaussian Mixture Models) - com probabilidades

```
# Prever com GMM
cluster = gmm.predict(novo_vinho_scaled)
probabilidades = gmm.predict_proba(novo_vinho_scaled)
print(f"Cluster previsto: {cluster[0]}")
print("Probabilidades de pertença a cada cluster:", probabilidades[0])
```

Nota sobre DBSCAN

Modelos como **DBSCAN** não são generalizáveis para novos dados — isto é, **não têm .predict()**. Para aplicar DBSCAN a novos dados, seria necessário reajustar o modelo incluindo os novos pontos.

Considerações finais:

- Certifica-te sempre de que a **ordem e escala das features** do novo dado corresponde exatamente ao treino.
- Modelos como K-Means e GMM são adequados para uso contínuo com novos dados.
- Para produção, guarda o modelo e o scaler com joblib ou pickle.

```
import joblib
# Guardar
joblib.dump(kmeans, "modelo_kmeans.pkl")
joblib.dump(scaler, "scaler.pkl")

# Reutilizar
kmeans = joblib.load("modelo_kmeans.pkl")
scaler = joblib.load("scaler.pkl")
```

Na secção mostramos como **utilizar os modelos de clustering** para tarefas comuns no mundo real, com base em dados não rotulados. As aplicações seguem uma lógica passo a passo desde a preparação até à interpretação dos resultados.

Passo 1: Carregar e explorar os dados

Ask ChatGPT B I Aa

```
import pandas as pd

# Carregar o dataset Wine Quality
df = pd.read_csv("winequality-red.csv", sep=';')
print(df.info())
print(df.describe())
```

Objetivo: Verificar a estrutura, tipo de variáveis, existência de valores em falta.

Passo 2: Preparar os dados

```
from sklearn.preprocessing import StandardScaler

# Selecionar apenas variáveis preditoras (excluindo 'quality')
X = df.drop(columns='quality')

# Normalizar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Objetivo: Garantir que as variáveis estejam na mesma escala para não influenciar indevidamente as distâncias.

Passo 3: Aplicar o modelo de clustering (K-Means)

```
from sklearn.cluster import KMeans

# Criar o modelo
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X_scaled)
```

Passo 4: Avaliar os resultados internamente

```
from sklearn.metrics import silhouette_score

sil_score = silhouette_score(X_scaled, labels)
print(f"Coeficiente de Silhueta: {sil_score:.2f}")
```

Objetivo: Avaliar quão bem os clusters estão definidos, com valores próximos de 1 indicando melhor separação.

Passo 5: Visualizar os agrupamentos

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='plasma')
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.title("Clusters Visualizados em 2D")
plt.show()
```

Passo 6: Relacionar com variáveis de interesse

```
# Adicionar as etiquetas de cluster ao DataFrame
clustered_df = df.copy()
clustered_df['cluster'] = labels

# Analisar a média da qualidade por cluster
print(clustered_df.groupby('cluster')['quality'].mean())
```

Objetivo: Verificar se os clusters coincidem com níveis diferentes de qualidade, mesmo sem o algoritmo saber disso previamente.

Desafio Final: Agrupamento de Clientes com Dados Reais

Objetivo: Aplicar tudo o que foi aprendido para resolver um problema real de segmentação.

Enunciado:

Foi-te entregue um dataset com informações anónimas sobre clientes de um retalhista online. O objetivo é **identificar grupos de clientes com comportamentos semelhantes** para personalizar estratégias de marketing.

O que deves fazer:

1. **Carrega o dataset clientes.csv** que contém colunas como: idade, salario_anual, frequencia_compra, gasto_mensal, tempo_como_cliente, entre outras.
2. **Explora os dados:** verifica valores ausentes, tipos de variáveis, estatísticas.
3. **Normaliza as variáveis numéricas.**
4. **Aplica pelo menos dois métodos de clustering** (ex: K-Means e DBSCAN ou GMM).
5. **Visualiza os resultados** com PCA.
6. **Analisa os perfis de cada cluster** e escreve conclusões (ex: cluster 1 são jovens com baixo gasto, cluster 2 são clientes antigos e fiéis, etc).

Entregáveis:

- Código funcional com todos os passos
- Gráficos de visualização
- Tabela descritiva por cluster (médias ou contagens por grupo)
- Texto interpretativo com sugestões práticas

Este desafio simula uma situação comum em ciência de dados aplicada a negócios reais. Não há uma única resposta certa — o que importa é **justificar bem cada decisão** e interpretar os resultados de forma crítica.